MRV System — Implementation Phases

Phase Dependencies

```
Phase 0 (Design & Spikes)
    │
    ▼
Phase 1 (Foundation & Infra) ──────────────────────────┐
    │                                                  │
    ├──► Phase 2 (Orgs & Users)                        │
    │       │                                          │
    │       ├──► Phase 3 (Templates) ◄── can parallel ─┤
    │       │       │                                  │
    │       │       ▼                                  │
    │       │   Phase 4 (Report Filling)               │
    │       │       │                                  │
    │       │       ▼                                  │
    │       │   Phase 5 (Submission & FMT Review)      │
    │       │       │                                  │
    │       │       ▼                                  │
    │       │   Phase 6 (Auditor Review)               │
    │       │       │                                  │
    │       │       ▼                                  │
    │       │   Phase 7 (PDF Export & Notifications)   │
    │                                                  │
    CI/CD + Testing + Security ── runs throughout ─────┘
```

Parallelization opportunities: - Phase 2 backend and Phase 3 backend can run in parallel (different domains, both depend only on Phase 1) - PDF export (Phase 7) can begin during Phase 6 using mock data - In-app notifications infrastructure starts in Phase 5 (when notification-worthy events first appear)

Phase 0: Design & Technical Spikes — Risk: MEDIUM

Goal: Finalize UI design, resolve unproven technology choices, and get client sign-off before implementation.

Duration: 1 sprint

0.1 Wireframes & UI Mockups (SOW Deliverable)

- ☐ Create Figma wireframes for all key screens:
  - ☐ Login / forgot password
  - ☐ Dashboard (per role: FMT, Country, Auditor)
  - ☐ Organization & user management (FMT admin)
  - ☐ Template builder (section editor, preview)
  - ☐ Report editor (section nav, rich text, tables)
  - ☐ Report review interface (comments panel, diff view)
  - ☐ Notification center
- ☐ Map Catalyst components to each wireframe element
- ☐ Client review and sign-off on wireframes

0.2 Technical Spike: Rich Text Editor

- [ ] Evaluate TipTap v2 with Catalyst/Tailwind styling
- [ ] Test table extension for structured data entry
- [ ] Test JSON serialization/deserialization round-trip
- [ ] Test content diff capability (for Phase 5 change tracking)
- [ ] Document findings and confirm TipTap as the choice

0.3 Technical Spike: PDF Generation

- [ ] Prototype PDF export using @react-pdf/renderer in Lambda
- [ ] Test rendering of TipTap JSON content to PDF
- [ ] Test table rendering in PDF
- [ ] Measure Lambda memory usage and cold start time
- [ ] Document findings and confirm library choice

0.4 Technical Spike: Change Tracking / Diff

- [ ] Research JSON diff algorithms for TipTap document format
- [ ] Prototype diff computation between two TipTap JSON documents
- [ ] Prototype visual diff rendering in React (insertions/deletions highlighting)
- [ ] Document approach and complexity assessment

Deliverables

- Figma wireframes reviewed and approved by client
- TipTap confirmed as rich text editor with proven capabilities
- PDF generation approach confirmed with acceptable performance
- Change tracking approach documented with feasibility confirmed

Acceptance Criteria

- Client has signed off on all wireframes
- Each spike has a working prototype demonstrating the capability
- All three technology decisions are finalized (no “or” in the architecture doc)

Phase 1: Project Foundation & Infrastructure — Risk: MEDIUM

Goal: Set up the full development environment, CI/CD pipeline, infrastructure, database, authentication, and frontend shell. A developer can build and deploy features after this phase.

Duration: 2 sprints

Prerequisites: Phase 0 complete (spikes resolved)

1.1 Project Scaffolding

- [ ] Initialize pnpm workspace monorepo (`pnpm-workspace.yaml`)
- [ ] Initialize SST v3 project (`sst.config.ts`)
- [ ] Set up TypeScript configuration (root `tsconfig.json` + per-package configs)
- [ ] Configure ESLint (flat config) + Prettier

- ☐ Create package structure: `core`, `db`, `functions`, `web`
- ☐ Set up Vitest for unit/integration testing (config in root + per-package)
- ☐ Add `lint`, `typecheck`, `test` scripts to root package.json

1.2 CI/CD Pipeline (SOW Requirement)

- ☐ GitHub Actions workflow: lint + typecheck + test on PR
- ☐ GitHub Actions workflow: deploy to `staging` on merge to main
- ☐ GitHub Actions workflow: deploy to `prod` on release tag (manual approval)
- ☐ SST stages configured: `dev`, `staging`, `prod`
- ☐ Environment-specific configuration (min/max ACU, deletion protection, etc.)

1.3 Infrastructure (SST v3)

- ☐ VPC: 2 public subnets (NAT Gateway), 2 private app subnets, 2 private DB subnets
- ☐ Security groups: Lambda SG, RDS Proxy SG, Aurora SG (see ARCHITECTURE.md section 6)
- ☐ VPC endpoints: S3 (gateway), Secrets Manager (interface)
- ☐ Aurora Serverless v2 PostgreSQL cluster (dev: 0.5-2 ACU)
- ☐ RDS Proxy for Lambda connection pooling
- ☐ Cognito User Pool with groups: `fmt`, `country`, `auditor`
- ☐ Cognito App Client (SRP auth flow)
- ☐ S3 bucket for file storage (block public access, SSE-S3 encryption)
- ☐ API Gateway HTTP API with JWT authorizer (Cognito)
- ☐ Next.js site deployment via SST `Nextjs` construct
- ☐ CloudFront distribution (managed by SST Nextjs)
- ☐ Secrets Manager for database credentials
- ☐ CloudWatch log groups for Lambda functions
- ☐ WAF with AWS managed rules on CloudFront

1.4 Database Schema & Tooling

- ☐ Set up Drizzle ORM with `postgres` driver
- ☐ Define all enums (org_type, user_status, template_status, report_status, etc.)
- ☐ Define schema: `organizations`, `users`, `countries`
- ☐ Define schema: `report_templates`, `template_sections`
- ☐ Define schema: `reports`, `report_sections`, `report_revisions`
- ☐ Define schema: `comments`, `change_records`
- ☐ Define schema: `notifications`, `audit_log`
- ☐ Generate initial migration with Drizzle Kit
- ☐ Create migration Lambda function (runs on deploy via afterDeploy hook)
- ☐ Create seed script: FMT organization + admin user + 2 sample countries
- ☐ Verify seed runs successfully in dev

1.5 API Foundation

- ☐ Lambda middleware: extract user context from JWT claims (sub, groups, organization_id)

- ☐ Lambda middleware: Zod request body validation
- ☐ Lambda middleware: structured JSON error responses (see ARCHITECTURE.md section 11.1)
- ☐ Lambda middleware: structured JSON logging (CloudWatch)
- ☐ RBAC helper in `packages/core/src/auth/` : `requireRole()` , `requireOwnership()`
- ☐ Data isolation helper: `scopeByOrganization(userId)` for query scoping
- ☐ Audit log helper: `logAction(userId, action, entityType, entityId, metadata)`
- ☐ Health check endpoint: `GET /health`
- ☐ CORS configuration on API Gateway

1.6 Authentication

- ☐ Cognito User Pool integration via SST (linked to API)
- ☐ API Gateway JWT authorizer configured
- ☐ `GET /auth/me` endpoint (returns user profile from DB, matched by cognito_sub)
- ☐ Post-confirmation Lambda trigger: create user record in DB when Cognito user is confirmed
- ☐ Unit tests for RBAC helpers and auth middleware

1.7 Frontend Foundation

- ☐ Initialize Next.js app with App Router
- ☐ Install and configure Tailwind CSS + Catalyst component library
- ☐ Set up app layout with Catalyst `SidebarLayout` :
  - ☐ Sidebar with navigation items (per-role visibility)
  - ☐ Header with user menu dropdown
  - ☐ Notification bell placeholder (wired in Phase 5)
- ☐ Auth pages: login, forgot password, reset password (using Amplify Auth + Catalyst forms)
- ☐ Auth context provider (wraps Amplify, provides user/role to components)
- ☐ Protected route wrapper (redirect to /login if unauthenticated)
- ☐ API client: fetch wrapper with `Authorization: Bearer` header, error handling, auto-retry on token refresh
- ☐ React Query (TanStack Query) provider configured
- ☐ Loading skeleton component (reusable)
- ☐ Error boundary with Catalyst-styled error page
- ☐ Empty state component (reusable)

1.8 Testing Foundation

- ☐ Vitest configured with test utilities for Lambda handlers (mock event, mock context)
- ☐ Test database setup: use separate schema or test database for integration tests
- ☐ Playwright configured for E2E tests (runs against staging)
- ☐ At least 1 integration test: health check endpoint
- ☐ At least 1 E2E test: login flow

Deliverables

- Working monorepo deployed to dev and staging
- CI/CD pipeline running on every PR and merge

- Database provisioned with schema and seed data
- Login/logout functional end-to-end
- Authenticated API calls working with role extraction
- Structured error handling and logging in place

Acceptance Criteria

- `pnpm test` passes all unit and integration tests
- `npx sst dev` starts local development successfully
- Merge to main auto-deploys to staging
- User can log in, see role-appropriate sidebar, and call `GET /auth/me`
- API returns structured JSON errors on invalid requests

Phase 2: Organization & User Management — Risk: LOW

Goal: FMT users can manage organizations, countries, users, and auditor assignments.

Duration: 1.5 sprints

Prerequisites: Phase 1 complete

2.1 Backend — Organizations API

- ☐ `GET /organizations` — list with cursor pagination, filter by `type`
- ☐ `POST /organizations` — create (FMT only, validate with Zod)
- ☐ `PUT /organizations/:id` — update name/status (FMT only)
- ☐ RBAC: only FMT group can access all org endpoints
- ☐ Unit tests for organization service
- ☐ Integration tests for organization endpoints

2.2 Backend — User Management API

- ☐ `GET /organizations/:id/users` — list users in org
- ☐ `POST /organizations/:id/users` — invite user:
  - ☐ Create Cognito user with temporary password
  - ☐ Assign to correct Cognito group (fmt/country/auditor based on org type)
  - ☐ Create user record in DB
  - ☐ Cognito sends invite email automatically
- ☐ `PUT /users/:id` — update profile/role/status (FMT or self)
- ☐ Deactivate user: set status to INACTIVE, disable Cognito user
- ☐ RBAC: FMT can manage all users; users can update own profile
- ☐ Unit tests for user service
- ☐ Integration tests for user endpoints

2.3 Backend — Country Management API

- ☐ `GET /countries` — list countries with org and auditor info
- ☐ `POST /countries` — create (link to existing COUNTRY org)

- ☐ `PUT /countries/:id` — update name
- ☐ `POST /countries/:id/assign-auditor` — assign/reassign auditor organization
- ☐ Validation: auditor org must be of type AUDITOR
- ☐ Unit tests for country service
- ☐ Integration tests for country endpoints

2.4 Frontend — Admin Pages

- ☐ Organizations list page — Catalyst `Table` with type badges, search, pagination
- ☐ Create organization — Catalyst `Dialog` with form (name, type dropdown)
- ☐ Organization detail page — org info + user list table
- ☐ Invite user — Catalyst `Dialog` with form (email, first name, last name, role)
- ☐ User status toggle (activate/deactivate) with confirmation
- ☐ Countries list page — Catalyst `Table` with assigned auditor column
- ☐ Create country — Catalyst `Dialog` with form (name, ISO code, link to org)
- ☐ Assign auditor — Catalyst `Select` dropdown of auditor organizations
- ☐ Loading skeletons for all list pages
- ☐ Empty states for no organizations / no users / no countries

Deliverables

- FMT can create and manage Country and Auditor organizations
- FMT can invite users (Cognito user created, invite email sent)
- FMT can assign auditors to countries
- All endpoints have RBAC enforcement and input validation

Acceptance Criteria

- FMT admin can create a Country org, invite a user, and that user receives an invite email
- FMT admin can assign an auditor to a country
- Country/Auditor users cannot access admin pages (403)
- All list pages support pagination and show appropriate empty states

---

Phase 3: Report Template Management — Risk: MEDIUM-HIGH

Goal: FMT users can create, edit, version, and publish ISFL Monitoring Report templates with rich text and table sections.

Duration: 2 sprints

Prerequisites: Phase 1 complete (can run in parallel with Phase 2 backend)

3.1 Backend — Template API

- ☐ `POST /templates` — create new template (DRAFT, version 1, generates template_group_id)
- ☐ `GET /templates` — list with pagination, filter by status (FMT: all, others: PUBLISHED only)
- ☐ `GET /templates/:id` — get template with all sections
- ☐ `PUT /templates/:id` — update metadata (name, description) — DRAFT only

- ☐ `DELETE /templates/:id` — delete draft template
- ☐ `POST /templates/:id/publish` — DRAFT → PUBLISHED, set published_at
- ☐ `POST /templates/:id/clone` — create new DRAFT version (copy sections, increment version, same template_group_id)
- ☐ Validation: cannot publish template with zero sections
- ☐ Unit tests for template service (including versioning logic)
- ☐ Integration tests for template endpoints

3.2 Backend — Template Sections API

- ☐ `POST /templates/:id/sections` — add section to draft template
- ☐ `PUT /templates/:id/sections/:sectionId` — update section
- ☐ `DELETE /templates/:id/sections/:sectionId` — delete section
- ☐ `PUT /templates/:id/sections/reorder` — batch reorder (accepts array of {id, sort_order})
- ☐ Validation: template must be DRAFT for section modifications
- ☐ Zod schemas for editor_config per editor_type (RICH_TEXT vs TABLE)
- ☐ Unit tests for section service
- ☐ Integration tests for section endpoints

3.3 Rich Text Editor Integration

- ☐ Install and configure TipTap v2 with extensions:
  - ☐ StarterKit (headings, bold, italic, lists, blockquote)
  - ☐ Table extension (for TABLE editor type)
  - ☐ Placeholder extension
- ☐ Style TipTap with Tailwind classes to match Catalyst design
- ☐ Toolbar component: heading levels, bold, italic, bullet list, numbered list, table insert
- ☐ JSON serialization: TipTap document ↔ JSON for API storage
- ☐ Test round-trip: type content → save JSON → reload → content preserved

3.4 Frontend — Template Builder

- ☐ Templates list page — Catalyst `Table`:
  - ☐ Columns: name, version, status (Badge), created date, actions
  - ☐ "New Template" button (FMT only)
  - ☐ Version history: group by template_group_id, show version lineage
- ☐ Template editor page:
  - ☐ Metadata form: name, description (Catalyst `Field` + `Input`)
  - ☐ Section list with drag-and-drop reordering (use `@dnd-kit/sortable`)
  - ☐ "Add Section" button
  - ☐ Per-section editor:
    - ☐ Title field
    - ☐ Instructions editor (TipTap rich text)
    - ☐ Editor type selector: Rich Text or Table
    - ☐ For TABLE type: column definition UI (add/remove columns, column name, type, required)

    - ☐ Sample/default data editor
  - ☐ Delete section with confirmation
- ☐ Publish confirmation dialog ("This will make version N available to countries")
- ☐ Template preview mode: read-only rendering as a country user would see it
- ☐ Clone template: "Create New Version" button on published templates
- ☐ Loading skeletons and empty states

Deliverables

- FMT can build report templates with ordered sections
- Sections support rich text (TipTap) and table editors
- Templates can be versioned (clone → edit → publish)
- Published templates are visible to country users

Acceptance Criteria

- FMT can create a template, add 3+ sections (mix of rich text and table), and publish it
- FMT can clone a published template, modify it, and publish as version 2
- Country users see only published templates in their template list
- Rich text round-trip: content entered in TipTap is preserved after save and reload
- Sections can be reordered via drag-and-drop

Phase 4: Report Creation & Filling — Risk: MEDIUM

Goal: Country users can create reports from published templates, fill each section, and validate completeness.

Duration: 1.5 sprints

Prerequisites: Phase 3 complete (published templates must exist)

4.1 Backend — Report API

- ☐ `POST /reports` — create report from template:
  - ☐ Copy template sections to report_sections with empty content
  - ☐ Set status to DRAFT, revision to 0
  - ☐ Validate template is PUBLISHED
- ☐ `GET /reports` — list reports, scoped by role (see RBAC matrix)
  - ☐ Cursor pagination, filter by status, country_id
- ☐ `GET /reports/:id` — get report with sections, template info, completion status
- ☐ `DELETE /reports/:id` — delete draft report (Country, DRAFT only)
- ☐ `PUT /reports/:id/sections/:sectionId` — save section content:
  - ☐ Optimistic locking via `version` field and `If-Match` header
  - ☐ Return 409 on version conflict
  - ☐ Auto-set `is_complete` based on content presence
  - ☐ Update `last_edited_by` and `last_edited_at`
- ☐ `POST /reports/:id/validate` — check all sections, return missing list
- ☐ Unit tests for report service (including optimistic locking)
- ☐ Integration tests for report endpoints

4.2 Frontend — Report Filling

- ☐ "New Report" page:
  - ☐ Select from published templates (Catalyst `Select` or card grid)
  - ☐ Create report → redirect to editor
- ☐ Reports list page — Catalyst `Table`:
  - ☐ Columns: template name, status (Badge), progress (% complete), last modified, actions
  - ☐ Status-specific actions (Edit for DRAFT, View for submitted)
- ☐ Report editor page:
  - ☐ Left sidebar: section navigation with completion indicators (checkmark / warning icon)
  - ☐ Main area: section title, instructions (read-only), and content editor
  - ☐ TipTap editor for RICH_TEXT sections (pre-filled with sample_data if available)
  - ☐ Table editor for TABLE sections (columns from editor_config, add/delete rows)
  - ☐ Auto-save with debounce (2s): "Saving..." → "Saved" indicator
  - ☐ Optimistic locking: show conflict dialog on 409
- ☐ Completeness check:
  - ☐ "Check Completeness" button: highlights incomplete sections in sidebar
  - ☐ Block submission if any section is incomplete
- ☐ Delete draft report with confirmation dialog
- ☐ Loading skeletons and empty states

4.3 Template Version Upgrade

- ☐ Backend: check if newer published version exists for report's template_group_id
- ☐ Backend: create new report from latest template, pre-fill matching sections by title/sort_order
- ☐ Frontend: notification banner on report editor when new version available
- ☐ Frontend: "Upgrade to Version N" button with confirmation
- ☐ Frontend: show what will be preserved vs. new/changed sections

Deliverables

- Country users can create reports from published templates
- Section editing with rich text and tables works correctly
- Auto-save with optimistic locking prevents data loss
- Completeness validation blocks submission of incomplete reports

Acceptance Criteria

- Country user creates a report, fills 3 sections, and sees auto-save work
- Completeness check correctly identifies empty sections
- Version conflict shows appropriate error (simulate with two tabs)
- Template upgrade preserves existing section content where possible

---

Phase 5: Report Submission & FMT Review — Risk: HIGH

Goal: Full submission → review → comment → revision cycle between country users and FMT.

Duration: 2.5 sprints

Prerequisites: Phase 4 complete

5.1 Backend — State Machine

- ☐ Implement report state machine in `packages/core/src/services/report-workflow.ts`
- ☐ Define valid transitions map (see ARCHITECTURE.md section 10.2)
- ☐ Guard conditions per transition (e.g., all sections complete for submit, open comments for revision request)
- ☐ `transitionReport(reportId, targetStatus, userId)` — validates and executes transition
- ☐ Auto-transitions: SUBMITTED → REVIEW_IN_PROGRESS on first reviewer access
- ☐ Audit log entry on every state transition
- ☐ Unit tests for every valid transition and every invalid transition (must reject)

5.2 Backend — Submission & Revision

- ☐ `POST /reports/:id/submit` — DRAFT → SUBMITTED_TO_FMT:
  - ☐ Validate all sections complete
  - ☐ Create report_revision snapshot (content_snapshot = all section content as JSON)
  - ☐ Increment current_revision
  - ☐ Create notification for FMT users
- ☐ `POST /reports/:id/request-revision` — FMT_REVIEW → FMT_REVISION_REQUESTED:
  - ☐ Validate at least 1 open comment exists
  - ☐ Accept optional message
  - ☐ Create notification for country users
- ☐ `POST /reports/:id/approve` — FMT_REVIEW → FMT_APPROVED:
  - ☐ Validate no open comments
  - ☐ Create notification for country users
- ☐ `POST /reports/:id/forward-to-auditor` — FMT_APPROVED → SUBMITTED_TO_AUDITOR:
  - ☐ Validate country has assigned auditor
  - ☐ Create notification for auditor users
- ☐ Resubmission: country re-submits from DRAFT (after FMT_REVISION_REQUESTED → DRAFT → SUBMITTED_TO_FMT)
  - ☐ Creates new revision snapshot
- ☐ Integration tests for full submission cycle

5.3 Backend — Comments API

- ☐ `GET /reports/:id/comments` — list with filters (section_id, status), cursor pagination
- ☐ `POST /reports/:id/comments` — create comment:
  - ☐ RBAC: FMT during FMT review states, Country can reply during revision states
  - ☐ Link to current revision_id
  - ☐ Thread support via parent_comment_id
- ☐ `PUT /comments/:id` — edit own comment (content only)
- ☐ `POST /comments/:id/resolve` — mark as RESOLVED, set resolved_by and resolved_at
- ☐ `POST /comments/:id/reopen` — reopen resolved comment

- ☐ Create notification when comment is added (notify report owner / reviewer)
- ☐ Unit tests for comment permissions per state
- ☐ Integration tests for comment CRUD

5.4 Backend — Change Tracking

- ☐ On resubmission, compute diff between previous and current revision:
  - ☐ For each section, compare content_snapshot[section_id] across revisions
  - ☐ Use JSON diff library (e.g., `jsondiffpatch`) for TipTap document comparison
  - ☐ Store change_records (from_revision, to_revision, previous_content, new_content)
- ☐ `GET /reports/:id/revisions/:revisionId/diff` — return change_records for a revision
- ☐ Unit tests for diff computation

5.5 Backend — Notifications Infrastructure

- ☐ Notification service: `createNotification(userId, type, title, message, link)`
- ☐ Batch create: notify all users in an organization
- ☐ `GET /notifications` — list own, filter by is_read, cursor pagination
- ☐ `GET /notifications/unread-count`
- ☐ `PUT /notifications/:id/read`
- ☐ `POST /notifications/read-all`
- ☐ Integration tests for notification endpoints

5.6 Frontend — Submission Flow

- ☐ Submit button on report editor (visible when all sections complete)
- ☐ Pre-submission summary dialog: list all sections with completion checkmarks
- ☐ Submit confirmation: “This will send the report to FMT for review”
- ☐ Post-submission: report becomes read-only with status badge
- ☐ Status timeline/history on report page

5.7 Frontend — FMT Review Interface

- ☐ Review queue page: Catalyst `Table` of reports with status SUBMITTED_TO_FMT / FMT_REVIEW_IN_PROGRESS
  - ☐ Columns: country, template, submitted date, revision #, status
- ☐ Report review page:
  - ☐ Section-by-section navigation (same sidebar as editor, read-only content)
  - ☐ Comments panel per section:
    - ☐ List existing comments (threaded)
    - ☐ Add new comment (Catalyst `Textarea` + submit button)
    - ☐ Reply to comment
    - ☐ Resolve / reopen toggles
  - ☐ Open comments count badge per section in sidebar
- ☐ “Request Revision” action: summary of open comments, confirmation dialog
- ☐ “Approve” action: confirmation dialog (only when no open comments)

- ☐ "Forward to Auditor" action: confirmation dialog (only when FMT_APPROVED)

5.8 Frontend — Revision Handling (Country)

- ☐ Revision indicator on reports list (badge: "Revision Requested")
- ☐ On opening revision-requested report:
  - ☐ Comments visible alongside each section
  - ☐ Sections become editable again
  - ☐ Changed sections highlighted (from change tracking)
- ☐ Reply to comments, resolve comments inline
- ☐ Resubmit button (same flow as initial submission)
- ☐ Revision history page: list of revisions with diff viewer

5.9 Frontend — Notification Center

- ☐ Notification bell in header with unread count badge
- ☐ Dropdown: recent notifications with links to relevant reports
- ☐ "Mark all as read" action
- ☐ Full notifications page with filters
- ☐ Polling: fetch unread count on page navigation (not WebSocket for MVP)

Deliverables

- Full submission → FMT review → revision → resubmission cycle working
- Section-level commenting with threads and resolve/reopen
- Change tracking between revisions with visual diff
- In-app notifications for all workflow events
- State machine enforcing valid transitions only

Acceptance Criteria

- Country submits report → FMT sees it in review queue
- FMT adds comments to sections, requests revision → Country is notified
- Country edits sections, resolves comments, resubmits → new revision created
- FMT approves and forwards to auditor → auditor is notified
- Invalid state transitions are rejected with 403/400
- Diff view shows what changed between revision 1 and revision 2

Phase 6: Auditor Review & Completion — Risk: LOW-MEDIUM

Goal: Auditors review reports forwarded by FMT, can request revisions, and mark reports complete.

Duration: 1 sprint

Prerequisites: Phase 5 complete

6.1 Backend — Auditor Workflow

- ☐ State transitions already implemented in Phase 5 state machine:

  - ☐ SUBMITTED_TO_AUDITOR → AUDITOR_REVIEW_IN_PROGRESS (auto on access)
  - ☐ AUDITOR_REVIEW → AUDITOR_REVISION_REQUESTED
  - ☐ AUDITOR_REVIEW → COMPLETED
  - ☐ AUDITOR_REVISION_REQUESTED → DRAFT (country re-edits)
- ☐ `POST /reports/:id/complete` — mark as COMPLETED:
  - ☐ Validate no open comments
  - ☐ Set completed_at timestamp
  - ☐ Create notification for country users
  - ☐ Create final revision snapshot
- ☐ RBAC: auditor can only access reports for countries assigned to their organization
- ☐ Integration tests for auditor workflow (submit → review → request revision → resubmit → complete)

6.2 Frontend — Auditor Review Interface

- ☐ Auditor dashboard: reports pending review (same Catalyst `Table` pattern as FMT)
  - ☐ Only shows reports for countries assigned to auditor's organization
- ☐ Report review page: reuse FMT review component with auditor-scoped permissions
  - ☐ Comment panel (same as FMT)
  - ☐ "Request Revision" action
  - ☐ "Mark as Complete" action with confirmation dialog
- ☐ Country revision handling for auditor comments (reuse Phase 5 revision UI)

6.3 Report Completion Side Effects

- ☐ Completed report is permanently read-only (no further edits)
- ☐ Completed status shown with green badge throughout UI
- ☐ PDF export button becomes available (preview of Phase 7)
- ☐ Audit log records completion event

Deliverables

- Auditor can review, comment, request revisions on assigned reports
- Auditor can mark reports as complete
- Full end-to-end workflow: Draft → FMT Review → Auditor Review → Complete

Acceptance Criteria

- Auditor sees only reports for their assigned countries
- Auditor cannot access reports for unassigned countries (403)
- Full cycle: country submits → FMT reviews → forwards to auditor → auditor completes
- Completed report is locked and shows as complete for all parties

---

Phase 7: PDF Export & Email Notifications — Risk: MEDIUM

Goal: Export completed reports as PDF. Send email notifications for key events.

Duration: 1.5 sprints

Prerequisites: Phase 6 complete

7.1 PDF Export

- ☐ Lambda function for PDF generation (1024 MB memory, 60s timeout):
  - ☐ Fetch report data with all sections and metadata
  - ☐ Render using @react-pdf/renderer:
    - ☐ Cover page: report title, country, template version, completion date, World Bank logo placeholder
    - ☐ Table of contents: list all sections
    - ☐ Per section: title, instructions, content (rich text rendered, tables formatted)
    - ☐ Footer: page numbers, "Generated on {date}"
  - ☐ Upload generated PDF to S3
- ☐ `GET /reports/:id/export/pdf`:
  - ☐ If PDF exists in S3, return pre-signed URL (1h expiry)
  - ☐ If not, trigger generation and return URL when ready
- ☐ Frontend: "Export PDF" button on completed reports
- ☐ Frontend: download link opens in new tab
- ☐ Unit test for PDF generation with mock data
- ☐ Integration test for export endpoint

7.2 Email Notifications (SES)

- ☐ Configure SES via SST `Email` construct
- ☐ SES domain verification for sending domain
- ☐ Email templates (HTML):
  - ☐ Report submitted (to FMT reviewers)
  - ☐ Revision requested (to country users)
  - ☐ Report approved / forwarded to auditor
  - ☐ Report completed (to country users)
  - ☐ New template version published
- ☐ Integrate email sending into existing notification service (Phase 5.5):
  - ☐ When creating in-app notification, also send email
  - ☐ Use SES SDK in notification service
- ☐ Bounce/complaint handling: CloudWatch alarm on SES bounce rate
- ☐ Integration test for email sending (use SES sandbox)

Deliverables

- PDF export for completed reports with professional layout
- Email notifications sent for all key workflow events
- Pre-signed S3 URLs for secure PDF downloads

Acceptance Criteria

- Completed report exports as a readable PDF with all sections and metadata
- PDF download link works and expires after 1 hour
- Email is received when report is submitted, revision requested, and completed

- Emails contain links back to the relevant report in the app

---

Phase 8: Polish, Security Hardening & UAT — Risk: LOW

Goal: Production readiness, UX polish, security audit, and user acceptance testing.

Duration: 1.5 sprints

Prerequisites: Phase 7 complete

8.1 UX Polish

- ☐ Audit all pages for consistent Catalyst component usage
- ☐ Loading skeletons on all data-fetching pages (verify)
- ☐ Error boundaries on all route segments
- ☐ Empty states for all list pages (verify)
- ☐ Form validation errors displayed inline (verify all forms)
- ☐ Responsive review: desktop-focused, functional on tablet widths
- ☐ Keyboard navigation: all interactive elements accessible via keyboard
- ☐ Browser support: Chrome, Firefox, Safari, Edge (latest 2 versions)

8.2 Security Audit

- ☐ RBAC enforcement audit: test every endpoint with wrong role (automated)
- ☐ Data isolation audit: test cross-org data access (Country A cannot see Country B)
- ☐ Input validation audit: verify Zod schemas on all endpoints
- ☐ Rich text sanitization: verify TipTap content is sanitized on output (no XSS)
- ☐ S3 bucket policy audit: verify no public access
- ☐ API Gateway throttling configured
- ☐ CORS: verify only frontend domain is allowed
- ☐ Dependency audit: `pnpm audit` for known vulnerabilities
- ☐ Secrets: verify no credentials in code or environment variables (all via Secrets Manager / SST links)

8.3 Testing Completion

- ☐ Unit test coverage: core business logic at >80%
- ☐ Integration test coverage: all API endpoints
- ☐ E2E test suite (Playwright):
  - ☐ Login and role-based navigation
  - ☐ FMT: create template → add sections → publish
  - ☐ Country: create report → fill sections → submit
  - ☐ FMT: review → comment → request revision
  - ☐ Country: address comments → resubmit
  - ☐ FMT: approve → forward to auditor
  - ☐ Auditor: review → mark complete
  - ☐ PDF export

- ☐ Performance check: API response time < 500ms for CRUD, < 3s for PDF generation

8.4 Production Deployment

- ☐ Production environment deployed (`npx sst deploy --stage prod`)
- ☐ Custom domain configured (Route 53 + ACM)
- ☐ WAF rules verified on production
- ☐ Aurora deletion protection enabled
- ☐ CloudWatch alarms configured (see ARCHITECTURE.md section 13.6)
- ☐ Backup verification: test Aurora point-in-time recovery
- ☐ Seed production data: FMT organization + admin user

8.5 User Acceptance Testing

- ☐ UAT environment = staging (or separate test stage)
- ☐ Client walkthrough of all user flows:
  - ☐ FMT admin flow
  - ☐ Country user flow
  - ☐ Auditor flow
- ☐ Client sign-off on UAT
- ☐ Bug fixes from UAT feedback

Deliverables

- Production-ready, security-hardened application
- Test coverage on all critical paths
- CI/CD pipeline deploying to staging and production
- Client UAT completed and signed off

Acceptance Criteria

- All E2E tests pass against staging
- Security audit: no unauthorized data access found
- `pnpm audit` shows no high/critical vulnerabilities
- Client has signed off on UAT
- Production environment is live and accessible